



8 Wege, das Kontextfenster von Claude Code optimal zu nutzen

Context Engineering statt Prompt Engineering

von Julian Ivanov

Warum das Kontextfenster so wichtig ist

Je mehr Kontext ein Sprachmodell auf einmal verarbeiten muss, desto schlechter wird die Qualität der Antworten. Dieses Phänomen heißt Context Rot und ist durch mehrere Studien belegt. Bei Claude Code ist das besonders relevant, denn das Kontextfenster füllt sich schnell: Es enthält deine gesamte Konversation, jede Datei, die Claude liest, und jede Befehlsausgabe.

Anthropic selbst betont an mehreren Stellen in ihrer Dokumentation, wie wichtig es ist, das Kontextfenster optimal zu nutzen. Es geht nicht nur darum, bessere Prompts zu schreiben, sondern um besseres Context Engineering. Das ist der eigentliche Hebel.

Hier sind acht konkrete Methoden, um das Kontextfenster effizient zu nutzen und dadurch deutlich bessere Ergebnisse zu bekommen.

1. Subagenten

Die mächtigste Methode. Subagenten haben ihr eigenes Kontextfenster und schonen damit den Hauptagenten.

Wie es funktioniert

Anstatt dass dein Hauptagent z.B. verschiedene Webseiten durchsucht, Vergleiche anstellt und Pro/Kontra-Listen erstellt, delegiert er diese Aufgabe an einen Subagenten. Der Subagent füllt sein eigenes Kontextfenster und gibt nur das Ergebnis zurück. Das Kontextfenster des Hauptagenten bleibt dabei nahezu unverändert.



Eingebaute Subagenten

Claude Code Guide: Beantwortet Fragen über Claude Code (nutzt Haiku).

Explore Agent: Durchsucht die gesamte Codebasis nach Dateien, Funktionen oder Strukturen.

General Purpose Agent: Für allgemeine Aufgaben wie Internetrecherchen.

Plan Agent: Wird automatisch im Planmodus genutzt.

Eigene Subagenten erstellen

Mit dem Befehl `/agents` kannst du eigene Subagenten anlegen. Beispiele:

Security Reviewer: Prüft den Code auf Sicherheitslücken und Best Practices.

Code Reviewer: Analysiert Code-Qualität und Konventionen.

UX Reviewer: Bewertet die User Experience deiner App.

Für jeden Subagenten kannst du festlegen, welche Tools er nutzen darf und welches Modell er verwenden soll.

Wann Subagenten nutzen?

Immer wenn ein hohes Volumen an Text verarbeitet wird: Tests, Audits, Dateisuche, Internetrecherchen. Du kannst Claude über die `CLAUDE.md` sogar anweisen, grundsätzlich als Manager zu arbeiten und Aufgaben an Subagenten zu delegieren.

2. Skills

Skills sind wiederkehrende Anweisungen, die Claude nur bei Bedarf abruft. Statt alle Informationen in die `CLAUDE.md` zu packen, lagerst du sie in Skills aus.

Warum Skills das Kontextfenster schonen

Jeder Skill hat einen YAML-Frontmatter mit einer kurzen Beschreibung. Nur diese Beschreibung wird in den Kontext geladen. Erst wenn Claude den Skill tatsächlich braucht, liest er die vollständige Skill-Datei aus. Dadurch verbrauchen Skills minimal Tokens.



Faustregel

In die CLAUDE.md: Nur Grundregeln, die immer gelten.

Als Skill: Alles, was du Claude mehr als zweimal erklärst.

Tipp: Skill Creator

Anthropic bietet einen offiziellen Skill Creator an. Dieser Skill erklärt Claude, wie gute Skills geschrieben werden, und evaluiert sie sogar mit Testbeispielen.

3. Planmodus

Bei großen Aufgaben (Apps, Websites, komplexe Skripte) solltest du immer zuerst den Planmodus nutzen, bevor Claude mit der Implementierung beginnt.

Warum?

Ohne Plan fängt Claude in eine Richtung an, die du vielleicht gar nicht wolltest. Dann iterierst du hin und her, Claude liest Dateien, schreibt Code, löscht ihn wieder. Ohne es zu merken, ist das Kontextfenster voll und die Qualität sinkt.

Der Trick: Plan speichern und frisch starten

Nachdem du mit Claude einen perfekten Plan ausgearbeitet hast:

1. Schreibe: "Speichere den Plan in einer Datei."
2. Starte einen neuen Chat mit /clear.
3. Schreibe: "Lies die Plan-Datei und setze den Plan um."

Damit hat Claude ein frisches Kontextfenster und den vollständigen Plan. Die gesamte Planungsdiskussion belastet den Kontext nicht mehr.

4. Präzise Prompts

Claude kann viel ableiten, aber keine Gedanken lesen. Je präziser du kommunizierst, desto weniger Tokens werden verschwendet.



Beispiele

Schlecht: "Mach das Dashboard schöner."

Besser: "Ändere die Schriftart von Arial auf Inter und nutze einen dunkelblauen Hintergrund."

Schlecht: "Es gibt einen Fehler im Formular."

Besser: "Beim Klick auf Speichern im Kontaktformular passiert nichts. Der Button scheint nicht mit der Submit-Funktion verbunden zu sein."

Weitere Tipps

@ für Dateien: Mit @ kannst du direkt Dateien referenzieren, die Claude berücksichtigen soll.

Screenshots: Lade Bilder oder Screenshots hoch, wenn du ein bestimmtes Design oder Layout möchtest. Visuell ist oft präziser als Text.

Interviewen lassen: Wenn du nicht weißt, was du sagen sollst, bitte Claude dich zu interviewen. Er stellt dann gezielte Fragen.

5. Die CLAUDE.md richtig nutzen

Die CLAUDE.md ist dein dauerhafter Kontext. Sie wird bei jeder Session geladen. Das bedeutet: Alles, was drinsteht, verbraucht immer Tokens.

Was die Studie sagt

Die ETH Zürich hat untersucht, ob CLAUDE.md Dateien wirklich helfen. Das Ergebnis:

Ohne CLAUDE.md: Baseline-Ergebnisse.

KI-generierte CLAUDE.md (/init): Tendenziell schlechtere Ergebnisse bei höheren Kosten.

Manuell geschriebene CLAUDE.md: Einzige Variante mit echtem Vorteil, aber nur wenn sie kurz und präzise ist.

Zu lange und ausführliche CLAUDE.md Dateien haben die Agenten eher verwirrt als unterstützt. Sie lasen mehr Dateien, liefen mehr Tests, aber lieferten nicht bessere Ergebnisse.



Empfehlung

Geh durch jede Zeile deiner CLAUDE.md und frage dich: Würde Claude Fehler machen, wenn diese Zeile nicht da wäre? Falls nein, raus damit. Anthropic empfiehlt unter 200 Zeilen. Nach der Studie kann man sie sogar noch kürzer halten.

Was gehört rein?

Ja: Codierungsregeln, Konventionen, Tech-Stack, Dinge die Claude nicht aus dem Code ableiten kann.

Nein: Projektbeschreibungen, Skill-Auflistungen, alles was Claude selbst herausfinden kann.

6. Kontexthygiene

Vier Befehle, die du unbedingt kennen und aktiv nutzen solltest:

`/context`

Zeigt dir, wie viel vom Kontextfenster belegt ist und wofür. Damit siehst du sofort, ob z.B. MCP-Server zu viele Tokens verbrauchen.

`/compact`

Komprimiert das Kontextfenster. Wichtig: Nutze diesen Befehl proaktiv bei ca. 70% Füllung, nicht erst wenn Claude automatisch bei 100% komprimiert.

Warum proaktiv? Bei automatischer Komprimierung hast du keinen Einfluss darauf, was Claude zusammenfasst. Wenn du selbst `/compact` aufrufst, kannst du angeben, was wichtig ist.

Beispiel: `/compact` – Behalte die geänderten Dateien und den aktuellen Plan.

`/clear`

Startet das Kontextfenster komplett neu. Nutze das nach jeder abgeschlossenen Aufgabe.



Der Progress-Trick

Bevor du /clear machst:

1. Schreibe: "Speichere den aktuellen Stand in einer progress.md."
2. Dann /clear für ein frisches Kontextfenster.
3. Schreibe: "Lies die progress.md und mach weiter mit [nächste Aufgabe]."

Das Gedächtnis von Claude Code liegt in Dateien. Speichere dir Informationen zwischen.

7. Richtige Modellauswahl

Nicht jede Aufgabe braucht das stärkste Modell. Opus denkt intensiv nach und verbraucht am meisten Tokens.

Übersicht

Opus 4.6: Für komplexe Aufgaben wie neue Features, Architekturentscheidungen, schwierige Bugs.

Sonnet: Beste Balance für die meisten Aufgaben. Alltag, Code schreiben, kleinere Features.

Haiku: Schnellstes und günstigstes Modell. Gut für Recherche, Dateisuche, einfache Fragen. Wird auch von den eingebauten Subagenten genutzt.

Wechseln: Mit /model kannst du jederzeit zwischen den Modellen wechseln.

8. CLI Tools statt MCP Server

Wenn du Claude Code mit externen Tools verknüpfen möchtest, gibt es drei Ansätze: API, MCP Server und CLI Tools. CLI Tools sind für das Kontextfenster am effizientesten.

Warum CLI Tools besser sind

MCP Server: Alle Toolbeschreibungen werden in den Kontext geladen, auch wenn du sie gerade nicht brauchst. Das frisst Tokens.

CLI Tools: Nichts wird vorgeladen. Claude öffnet das Terminal, gibt einen Befehl ein und bekommt eine Ausgabe. Wie Skills werden sie nur genutzt, wenn sie gebraucht werden.



Claude kennt die meisten CLI Tools bereits aus seinem Training (curl, git, docker). Bei neuen Tools ruft Claude einfach das Hilfe-Kommando auf (z.B. --help) und versteht sofort, wie das Tool funktioniert.

Faustregel

Wenn es für ein externes Tool sowohl einen MCP Server als auch ein CLI Tool gibt, nutze das CLI Tool. Installiere MCP Server nur auf Projektebene, nicht global, damit sie nicht in jedem Projekt Tokens verbrauchen.

Zusammenfassung

Methode

1. Subagenten
2. Skills
3. Planmodus
4. Präzise Prompts
5. CLAUDE.md optimieren
6. Kontexthygiene
7. Modellauswahl
8. CLI statt MCP

Kernidee

Aufgaben delegieren, Hauptkontext schonen
Wissen auslagern, nur bei Bedarf laden
Erst planen, dann mit frischem Kontext umsetzen
Klar kommunizieren, weniger Iterationen
Kurz halten, nur was Claude nicht ableiten kann
Proaktiv komprimieren, Progress speichern
Nicht immer Opus, Sonnet reicht oft
Kein Vorlade-Overhead, nur bei Bedarf

Die goldene Regel

Das Kontextfenster ist die wichtigste Stellschraube bei der Arbeit mit Claude Code. Halte es sauber, nutze Subagenten und Skills, komprimiere proaktiv und starte regelmäßig frisch. Der Unterschied ist spürbar.